```
  when:
            def proposedTrainTimes = itineraryService.
                 findNextDepartures("Parramatta", "Town Hall", at(8,00));

  then:
            proposedTrainTimes == [at(8,02), at(8,11),
                                   at(8,14), at(8,21)]
}
```

You now expect the itinerary service to return four times.

This, in turn, helps you isolate the code that needs to change in the `ItineraryService` class. From here, you'll be in a much better position to update the code correctly.

For larger changes, more work will obviously be involved. But the principle remains the same for modifications of any size. If the change request is a modification of an existing feature, you need to update the automated acceptance criteria to reflect the new requirement. If the change is a bug fix that your current acceptance criteria didn't catch, then you need to first write new automated acceptance criteria to reproduce the bug, then fix the bug, and finally use the acceptance criteria to demonstrate that the bug has been resolved. And if the change is big enough to make existing acceptance criteria redundant, you can delete the old acceptance criteria and write new ones.

2.6 Summary

In this chapter, you learned about the overall BDD project lifecycle. In particular, you saw the following:

- Understanding the underlying business objectives of a project lets you discover features and stories that can deliver these business objectives.
- Features describe functionality that will help users and stakeholders achieve their goals.
- Features can be broken down into stories that are easier to build and deliver in one go.
- Concrete examples are an effective way to describe and discuss features.
- Examples, expressed in a semi-structured "Given ... When ... Then" notation, can be automated in the form of automated acceptance criteria.
- Acceptance criteria drive the low-level implementation work and help you design and write only the code you really need.
- You can also use the BDD-style "Given ... When ... Then" structure in your unit tests.
- The automated acceptance criteria also document the features delivered, in the form of living documentation.
- Automated acceptance criteria and BDD-style unit tests make maintenance considerably easier.

In the following chapters, we'll discuss each of these themes in much more detail, and we'll also look at how the approaches we discuss can be put into practice using different tools and technologies.